

AI-агенты и Multi-Agent Systems

AI Архитектор



Проверить, идет ли запись

Меня хорошо видно & слышно?





Илья Ящук

Data Scientist

- 5 лет опыта в Reliability Engineering, Condition Monitoring
- Кандидат технических наук (предиктивная аналитика)
- Анализ данных и разработка ML-моделей
- MLOps в RenMoney (FinTech)



@ilia_iashchuk



Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в учебной группе **#канал
группы**



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Маршрут вебинара

Знакомство

Введение в AI агенты

Reasoning паттерны для агентов

Мультиагентные системы

Практика



Цели вебинара

К концу занятия вы сможете

1. Спроектировать AI-агент с архитектурой, подходящей именно для вашего сценария

2. Выбирать когда вам подойдет один AI-агент, а когда потребуется мультиагентная система

3. Проектировать мультиагентные системы, выбирая подходящий сценарий взаимодействия агентов в них

Смысл

Для чего вам это уметь

Проектировать AI-агенты – за ними будущее



**Пользуетесь ли вы уже
какими либо AI-агентами**



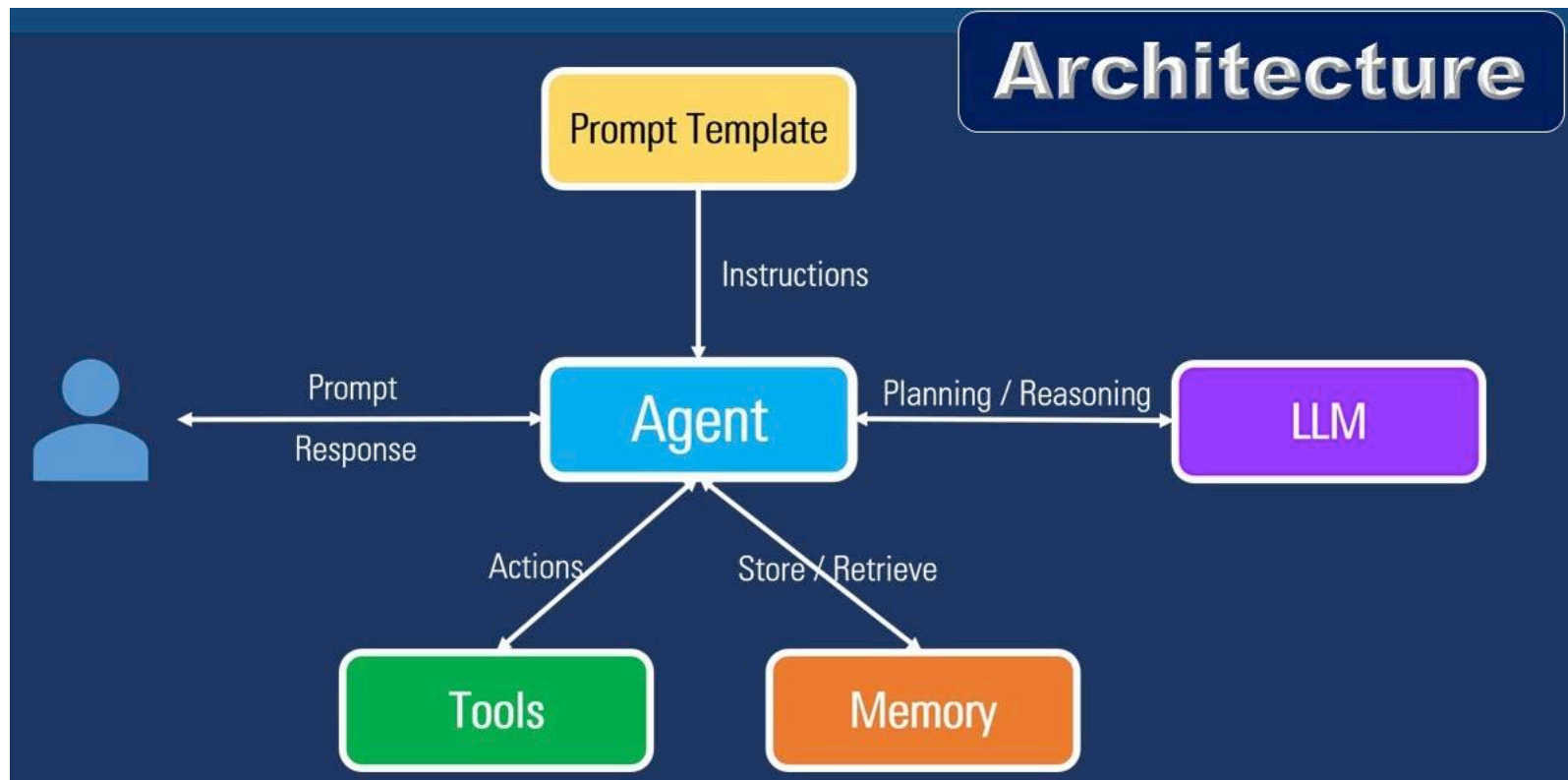
Введение в AI агенты

AI агенты

- Автономное программное обеспечение, способное воспринимать поставленную цель и выполнять многошаговые задачи для достижения этой цели
- Использует возможности LLM для размышлений, построения плана действий, генерации контента и вызова внешних API
- Использует память для хранения контекста
- Имеет ограниченный набор инструментов (tools), которые может использовать для выполнения поставленной задачи
- Взаимодействует с другими агентами и системами

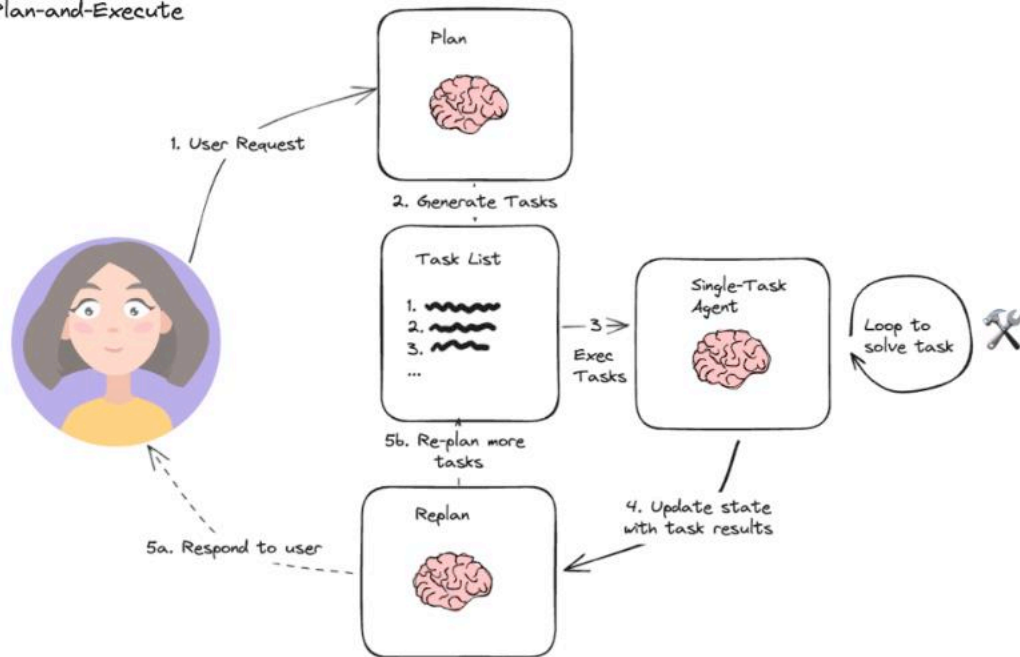


AI агенты



Пример простого AI-агента

Plan-and-Execute



LLM для агентов

- Не все LLM одинаково хорошо подходят для использования в AI-агентах
- Поддержка Function Calling
- Большой контекст (100k+)
- Часто предпочтительно использовать reasoning модели
- Leaderboard - <https://huggingface.co/spaces/galileo-ai/agent-leaderboard>



Память для AI-агентов

- Краткосрочная – хранение в рамках одной пользовательской сессии (история диалога, состояние и тд)
- Сообщения обычно хранятся и передаются типизированными – system, user, assistant, tool
- Различные стратегии для управления краткосрочной памятью:
 - очистка контекста (удаление старых сообщений)
 - суммаризация
 - запрос из внешнего хранилища (RAG-like)
- Долгосрочная – хранение информации вне сессии, глобальные знания агента



Архитектура агентов

Chain-of-thoughts

- Подход к составлению промта для Llm
- Позволяет улучшить качество ответов в основном на запросах, требующих сложной логики, а также на математических операциях
- Добавление к промту примеров рассуждений либо фразу, побуждающую к рассуждениям (Zero-Shot CoT)
- Использование инструментов либо невозможно, либо очень ограничено

Q: Jack has two baskets, each containing three balls. How many balls does Jack have in total?

A: One basket contains 3 balls, so two baskets contain $3 * 2 = 6$ balls.

Q: {QUESTION}

A:

Chain-of-thoughts. Пример

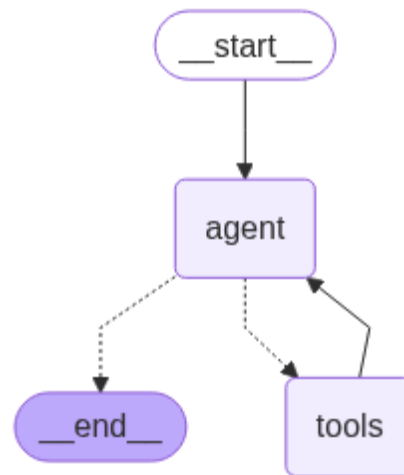
```
### System Instructions:
1. You are an **AI-powered travel planner** that helps users organize trips by p
2. You **always** follow a structured step-by-step Chain-of-Thought (CoT) reaso
3. Finally, you will create a a daily itinerary with the details for each day, f

### Example Interaction:
#### **User Query:**
"I want to plan a 5-day trip to Tokyo on a budget. Can you help?"
<reasoning>
Step 1: Understand User Preferences
- Destination: Tokyo
- Duration: 5 days
- Budget: Budget-friendly trip
Step 2: Plan Flights & Accommodation
- Search for **affordable flight options** from the user's location.
- Suggest **budget hotels, capsule hotels, or Airbnb options**.
Step 3: Create a Day-by-Day Itinerary
- **Day 1:** Arrive in Tokyo, explore Shinjuku & try local ramen.
- **Day 2:** Visit Asakusa & Senso-ji Temple, explore Akihabara.
- **Day 3:** Take a day trip to Mt. Fuji or Hakone.
- **Day 4:** Experience teamLab Planets, Shibuya Crossing, and Harajuku.
- **Day 5:** Shopping in Ginza, return flight.
Step 4: Budget Breakdown
- Flights: ~$600
- Accommodation: ~$50/night (capsule hotel)
- Daily meals: ~$30/day
- Transport: ~$10/day (Tokyo Metro Pass)
- Activities: ~$50 total
- **Total Estimated Budget:** ~$1,000
Step 5: Packing List & Travel Tips
- **Essentials:** Passport, travel adapter, Suica card for public transport.
- **Weather Considerations:** Pack light jackets if traveling in spring or fall.
- **Local Customs:** Cash is commonly used in Japan, carry some yen.
</reasoning>
<response>
"Here's a detailed 5-day **budget-friendly** Tokyo itinerary with flight, stay,
Please specify the final itinerary here.
</response>
```



ReAct

- Подход к построению логики AI агента
- Чередование мыслей (thoughts) с действиями (acts)
- Оценка состояния после выполнения действия и изменения состояния (observe)
- Позволяет эффективно использовать внешние инструменты
- Вызов внешних инструментов выполняется последовательно, по мере «рассуждений»
- Достаточно дорог по потреблению токенов за счет многократных вызовов llm
- Достаточно высокий latency



ReAct. Пример

```
### System Instructions:
1. You are a customer service AI assistant that helps users track their orde
2. You have access to an API function called 'get_order_status' that retrieves l
3. Follow the ReAct Think → Act → Observe → Respond process. You will alw
4. If there is a matching function, write down the specification for the func
```

```
### Available Function:
You can call the following function when needed:
tools = [{
  "name": "get_order_status",
  "description": "Retrieves the status of a customer's order, including current
  "parameters": {
    "order_id": {
      "type": "string",
      "description": "The unique identifier for the order."
    }
  }
}]
---
```

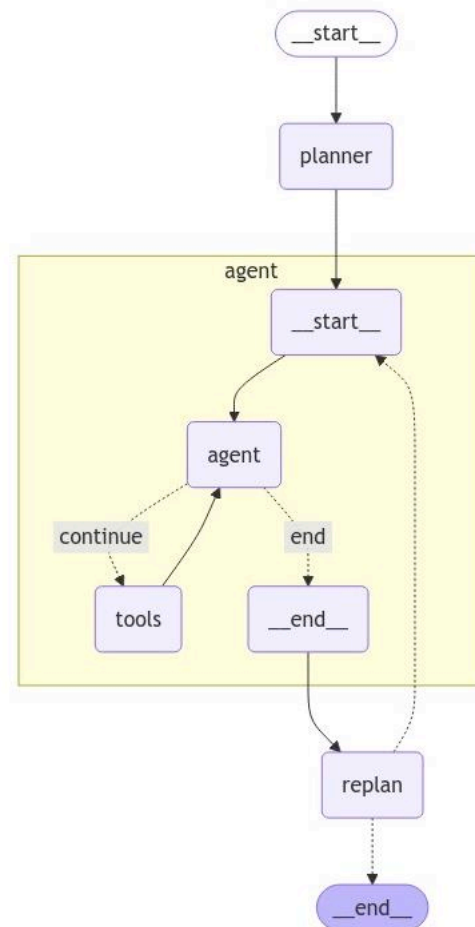
```
### Example Interaction:
#### User Query:
> "Where is my order #12345? I placed it last week."
<reasoning>
> "To provide an accurate update, I need to check the order status using the ord
> Action: Call 'get_order_status(order_id="12345")'
</reasoning>
<response>
```json
{
 "name": "get_order_status",
 "parameters": {
 "order_id": "12345"
 }
}
...
</response>

```



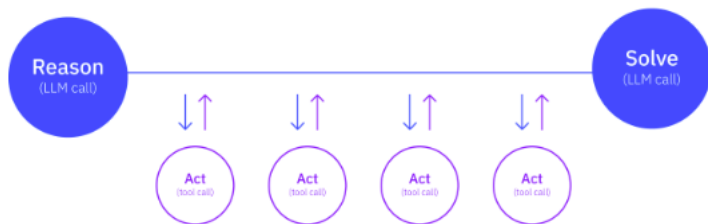
# Plan-and-Execute

- Подход к построению логики AI агента
- Отдельный этап предварительного планирования всех предполагаемых этапов, которые потребуются для решения задачи
- Выполнение всех запланированных команд последовательно
- Перепланирование, если цель не достигнута



# ReWOO (reasoning without observation)

- Подход к построению логики AI агента
- Отдельный этап предварительного планирования всех предполагаемых этапов, которые потребуются для решения задачи
- Выполнение всех запланированных команд, зачастую полностью или частично параллельно
- Отдельный этап агрегации результатов выполнения команд, перед отдачей ответа пользователю



# ReWOO. Пример

## ### \*\*System Instructions:\*\*

You are an **AI-powered RFP drafting assistant** that helps organizations create well-structured Requests for Proposals (RFPs). You follow the **ReWOO framework**, which consists of **three steps**:

1. **Planner** - Generate a structured plan for the RFP, including tool-use signatures.
  2. **Worker** - Execute all required tool calls based on the plan and store results.
  3. **Solver** - Combine the tool results with the plan to generate a finalized RFP.
- Your goal is to **maximize efficiency** by generating all tool calls in a single pass before execution.

---

## ### \*\*Tool Available:\*\*

```
```json
{
  "name": "generate_rfp_section",
  "description": "Generates a specific section of an RFP based on provided parameters.",
  "parameters": {
    "section": {
      "type": "string",
      "description": "The RFP section to generate (e.g., 'Introduction', 'Scope', 'Requirements')."
    },
    "industry": {
      "type": "string",
      "description": "The industry for which the RFP is being written (e.g., 'Healthcare', 'IT', 'Construction')."
    },
    "custom_requirements": {
      "type": "string",
      "description": "Additional details or specific requirements to include in the section."
    }
  }
}
```

You're in step 1. Let's just generate the instructions to call the tool in JSON format. No extra text.

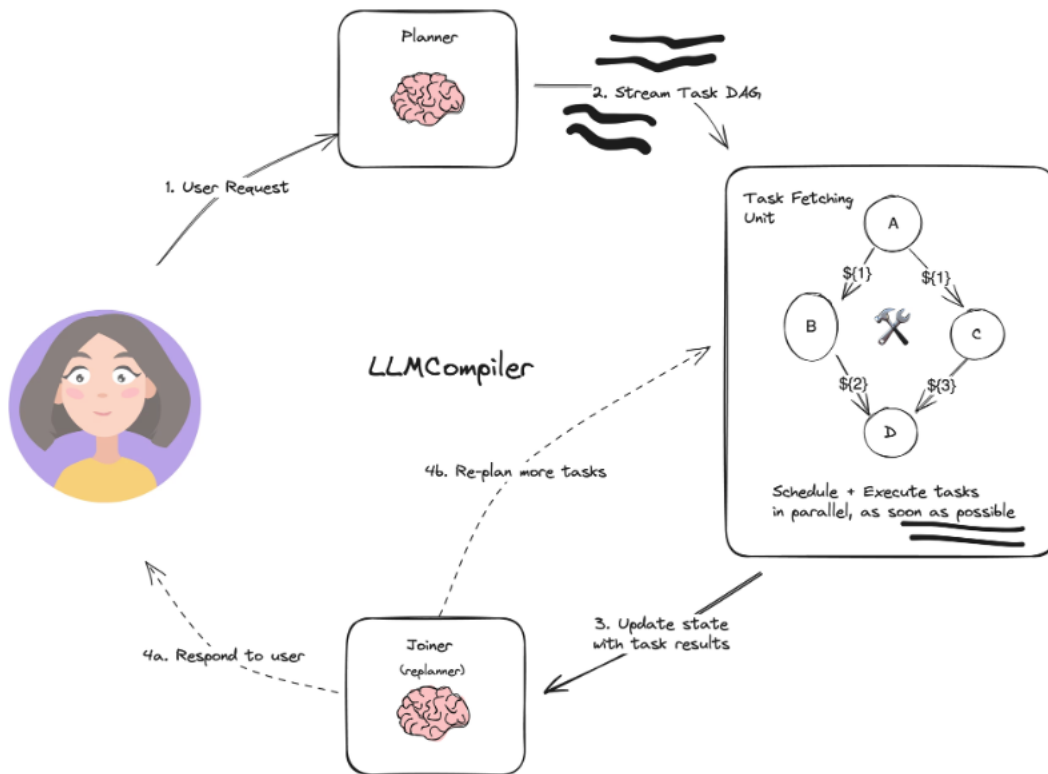


LLM Compiler

- Подход к построению логики AI агента
- Более продвинутый вариант паттерна ReWOO
- Задачи планируются в виде направленного ациклического графа
- Выполнение задач начинается по мере выполнения пререквизитов для каждого из инструментов
- Отдельный этап агрегации результатов выполнения команд, перед отдачей ответа пользователю
- Ссылка на ref - <https://arxiv.org/abs/2312.04511>



LLM Compiler



Плюсы и минусы. CoT / ToT

Плюсы:

- Простота применения, по сути простой промптинг
- Снижение уровня галлюцинаций
- Понятный путь достижения результата

Минусы:

- Невозможно нормально использовать внешние инструменты
- Галлюцинации все равно имеют место быть
- Понятный путь достижения результата
- Недостаточная гибкость

Когда стоит применять:

- Не требуется использование внешних инструментов или максимум один
- Решение логических и математических задач



Плюсы и минусы. ReAct

Плюсы:

- Полноценное взаимодействие с внешними инструментами
- Хорошее качество за счет постоянной оценки текущего состояния
- Возможность использования не самых мощных моделей за счет необходимости планирования только ближайшего шага

Минусы:

- Высокое потребление токенов и высокое латенси из за большого количества вызовов моделей
- Сложно использовать несколько разных моделей для разных задач
- Последовательное выполнение всех этапов

Когда стоит применять:

- Задачи, где последовательное выполнение задач естественно (веб-поиск, взаимодействие с пользователем)



Плюсы и минусы. Plan-and-Execute

Плюсы:

- Полноценное взаимодействие с внешними инструментами
- Меньшее потребление токенов и ниже латенси за счет возможности паралельного выполнения задач
- Возможность использовать менее дорогие модели для этапа выполнения задач

Минусы:

- Низкая эффективность в случаях, если план составлен не оптимально
- Высокие требования к качеству модели, генерирующей план выполнения

Когда стоит применять:

- Задачи с большим горизонтом планирования



Плюсы и минусы. ReWOO / LLM Compiler

Плюсы:

- Полноценное взаимодействие с внешними инструментами
- Меньшее потребление токенов и ниже латенси за счет возможности параллельного выполнения задач
- Возможность использовать менее дорогие модели для этапа выполнения задач

Минусы:

- Низкая эффективность в случаях, если план составлен не оптимально
- Высокие требования к качеству модели, генерирующей план выполнения

Когда стоит применять:

- Задачи, где этапы выполнения естественным образом хорошо параллелятся (генерация контента и схожие)



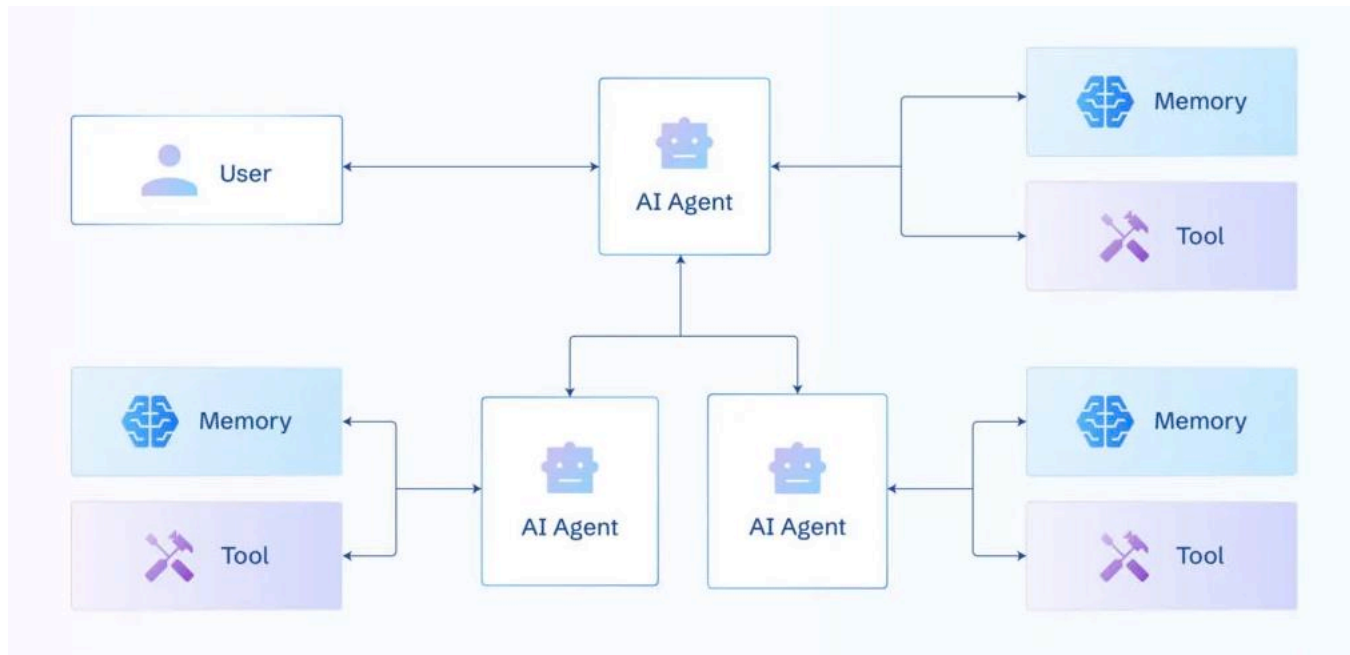
Мультиагентные системы

Когда одного агента недостаточно

- Сложные процессы, подразумевающие использование большого количества инструментов – один агент может плохо выбирать подходящий инструмент, если их много (4-5+)
- Сложная доменная область, требующая использования специализированных моделей для приемлемого качества
- Необходимость сочетания различных архитектурных подходов в разных задачах
- Желание и возможность иметь graceful degradation
- Удобство добавления нового функционала к уже существующему процессу
- Возможность параллельного выполнения

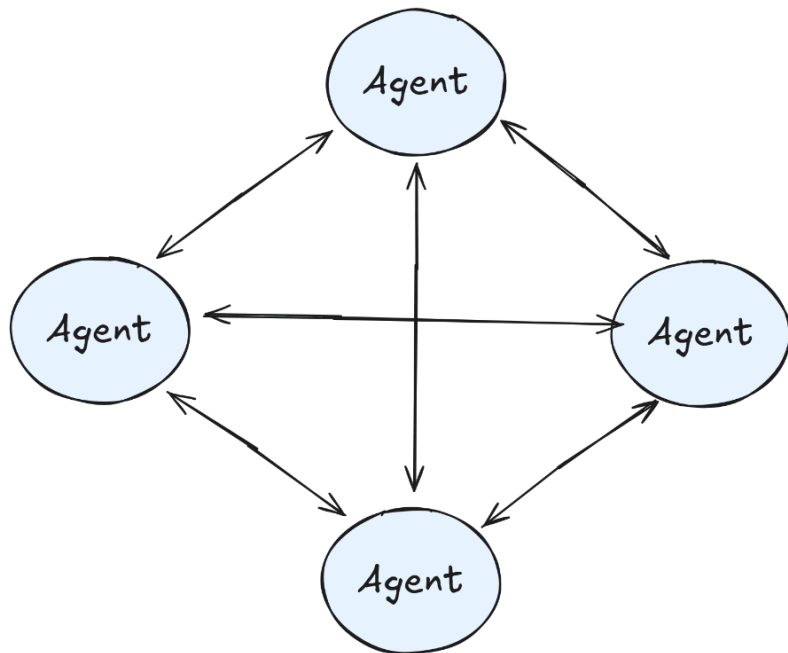


Мультиагентные системы

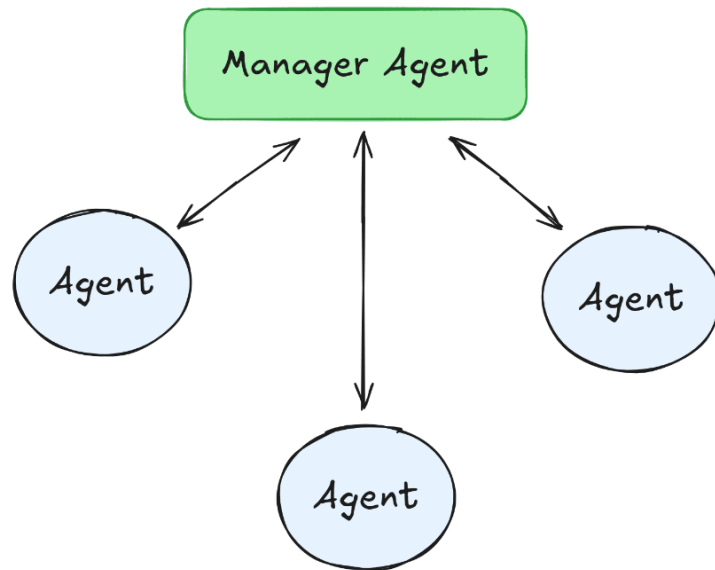


Мультиагентные системы

Network Architecture



Supervisor Architecture



Иерархия / Supervisor

- Один выделенный агент, который координирует вызовы всех остальных
- Весь контекст обычно хранит только координатор, остальным агентам передается только ограниченный контекст
- Хорошо подходит для средних / больших задач, и часто избыточен на небольших проектах



Network multiagent systems

- Все агенты равноправны и динамически определяют нужен ли вызов другого для решения задачи
- Пользователь может обращаться к любому из агентов, а не только к какому-то определенному
- Хорошо подходит для небольших задач, на средних и больших возникают проблемы с обнаружением других агентов и сложностью системы
- Весь контекст обычно шарится между всеми агентами



**Подведём
итоги занятия**

Цели вебинара

Теперь вы сможете

1. Спроектировать AI-агент с архитектурой, подходящей именно для вашего сценария

2. Выбирать когда вам подойдет один AI-агент, а когда потребуется мультиагентная система

3. Проектировать мультиагентные системы, выбирая подходящий сценарий взаимодействия агентов в них

Смысл

Для чего вам это уметь

Проектировать AI-агенты – за ними будущее

